

Project Objectives

This is a solo project that involves creating functions to combine and summarize data.

Setting Things Up

Repo Settings

The first step is to create a github repo and link it with an RStudio project. Push up those changes so the .Rproj file exists. By using an R project you can stick to using relative file paths for everything. **All project work should be done within this repo. You should be committing often so we can see your work develop (at least 6 substantial commits where we can see your work evolving). If you do not do this, you will lose substantial credit (up to 50 points).**

On your github project repo you should go into the settings and enable github pages. We'll output our files to a docs folder, similar to how we've done our homework assignments.

Overall Goal

Our goal is to write functions that will manipulate and process data sets that come from a census API. We'll create *generic* functions to automatically summarize and plot certain returned data. Lastly, we'll write up a document via quarto (just like we did with the homework) to describe our thought process and give examples of using the functions. The website created from this document is what you will ultimately turn in.

You must have a narrative throughout the document. This is often a place where people lose points on their first project. A narrative means you should have a story or thread that flows throughout the document. Things like an introduction about what you are doing, discussion of what you are about to do, what the results mean, etc. Don't lose these easy points!

Data

We are going to query the Public Use Microdata Sample (PUMS) Census API. This API gives pseudo *person-level* data. There is no need to obtain an API key but you can if you'd like.

Now there is an overwhelming amount of information about how to deal with census data and the different APIs there are. It is incredibly easy to go down a rabbit hole with older information or alternative ways to do things. **I hope to give you all the resources/links you need below!**

- [Information about the PUMS API](#)
 - We aren't going to worry about Puerto Rico data so focus on the basic PUMS information
 - There is data from many different years. We'll focus on 2021, 2022, 2023, and 2024 only.
 - Under each year, you can see example calls to the API. For instance,
`https://api.census.gov/data/2024/acs/acs1/pums?get=SEX,PWGTP,MAR&for=state:\*&SCHL=24&key=`
- The [list of variables](#) for the 2024 version is really useful
 - This page gives the values the variables can take on if you click on their name.
 - You can change the year in the URL to get a data dictionary for other years (again 2021, 2022, 2023, and 2024 are the only years we will care about)
- We'll want to allow the user to choose certain variables to return and choose certain conditions on what is returned (such as certain states - specifics given below)

- A [menu based explorer](#) does exist where you can use menus to get a certain subset and variables and then see API calls (by going to the ‘download’ tab and it is at the bottom). If you are struggling with the above, this may help but hopefully isn’t necessary.

Part 1: Data Processing

Parsing Variable Values We’re going to start by downloading and parsing the JSON files with the variable descriptions for each year.

- [2021 data](#), [2022 data](#), [2023 data](#), [2024 data](#)
- The only variables we want to parse are:
 - AGE, GASP, GRPI, JWAP, JWDP, JWMNP, PWGTP, FER, HHL, SCH, SCHL, SEX, ST (for 2021, 2022)/STATE (for 2023, 2024), REGION, and DIVISION
- Write a function to `GET()` the URL, parse the JSON, and pull out a list of the variables with their values. (`BaseR` list manipulations is the easiest way to go here!)
- Obtain one list for each year
- Save the lists in an `.rds` file. Save this `.rds` file in your repo under a `data` folder

Parsing API Data Alright, so we’ll want to write a function to query the PUMS API and return a nicely formatted tibble. When we query the API via our usual process (see notes), the data comes out pretty decently!

- Start by getting the usual process to work with a given URL
- Next, write a *helper* function to take what is returned by `GET()` and turn it into a nice `tibble`.
- Now we’ll wrap this into a function that also parses the data. Write a function to query the API that allows the user to change the following items:
 - Year of survey (2024 as default). Only a single value here
 - * Check that a valid value was given (2021, 2022, 2023, or 2024)
 - Specify the numeric variables to be returned (AGE and PWGTP as default). PWGTP should always be returned and one numeric variable other than PWGTP must be returned (check on this in your function).
 - * Options for the user should be AGE, GASP, GRPI, JWAP (time), JWDP (time), and JWMNP
 - * Your function should turn variables into numeric values or time (the `lubridate` packages has time formats) values (use the middle of the time period) where appropriate. This might require you to write a helper function or two to parse the strings! **This is one place I’m perfectly happy for you to use LLMs in solving.** Feel free to use them to figure out a good function for parsing the time strings and returning the midpoint as a time value. For missing time values, assign the value an ‘NA’.
 - Specify the categorical variables to be returned (SEX as default). One categorical variable must be returned (check on this in your function).
 - * Options for the user should be FER, HHL, SCH, SCHL, and SEX
 - * Your function should turn variables into factors with appropriate levels, where it makes sense. To do this, pull in the appropriate year’s variable levels from your `.rds` file.

- If the user specifies a variable not included in the above two lists, or does not have the required variables noted above, return an error.
- **Specify the geography level:** Either Region, Division, or State (with the default geography level of State). Note: We aren't including the "All" geography level simply due to the size of the data that is returned. Also, we are just choosing one of these to subset on for a given API call, not multiple at once.
 - * If the user specifies multiple values for geography, specifies "All", or provides a value that is not Region, Division, or State, return an error.
- Specify the specific region, division, or state to subset the data on
 - * You should specify a default level for state, division, or region if the user doesn't specify one (defaults of your choosing). You might set the value to NULL in the function definition and, if it is NULL in the function call, use a default value. However, feel free to do this how you'd like!
 - * The allowable values for the geographies are in the `.rds` file you made earlier
 - * The user should also be able to input an 'all' option here that would return all of values for that geography (for instance, all states)
- Lastly, add this line of code before returning your tibble:

```
class(your_tibble) <- c("census", class(your_tibble))
```

Combining Years

- Write a function that allows the user to specify multiple years of survey data (and all the other options above)
 - This function should call your single year function as many times as needed and then combine the data into one final tibble (a year variable should be included in this final tibble)

Part 2: Writing a Generic Function for Summarizing

The rows in our tibbles don't generally correspond to a single person! The `PWGTP` variable actually represents the number of (people) observations there should be for a particular row.

- We could create a new tibble with each row replicated `PWGTP` times and then the tibble would represent our usual type of data set.
- However, this would create a large tibble in some cases!
- Instead, we'll just have to be careful when summarizing the data.

Generic Functions We briefly discussed the ideas around object oriented programming and method dispatch. That is, we discussed why `plot(iris)` and `plot(exp)` give different types of plots. They look at the `class` of `iris` and the `class` of `exp` and the appropriate plotting function is called.

```
#Run these in your console
plot.function #what is used for a class = function
getS3method("plot", "data.frame") #what is used for a class = data frame
```

The generic function is `plot()`. Notice that it calls `UseMethod("plot")` and that is how the appropriate plotting function is determined.

```
plot
```

```
## function (x, y, ...)  
## UseMethod("plot")  
## <bytecode: 0x00000170076219a0>  
## <environment: namespace:base>
```

As we have added a `census` class to our tibble, we can write our own custom `summary()` function! We simply write our function as follows:

```
summary.census <- function(...){...}
```

Then we can call the function via `summary(our_object)` and it knows, based on the class, to use our `census summary()` function.

Numeric Summaries For the `census` summary method, let's write a function that produces means and standard deviations for our numeric variable(s) (ignore the time variables) and counts for our categorical variable(s) (year should be treated as categorical).

- This function should take three arguments: the census tibble (which may have multiple years), the numeric variable(s) to summarize, the categorical variable(s) to summarize.
- By default, it should summarize all numeric variables (other than `PWGTP`) and all categorical variables in the tibble. However, the user should be able to specify the variables they'd like to summarize if they'd like.
 - To find a sample mean for a numeric variable, we would do something like this:

```
sum(numeric_vector*weight_vector)/sum(weight_vector)
```

- To find a sample standard deviation:

```
sqrt(sum(numeric_vector^2*weight_vector)/sum(weight_vector)-sample_mean^2)
```

- Return the values as a named list

Graphical Summaries Similarly, let's create a generic `plot()` function for a `census` class tibble. Require the user to specify one categorical variable (which can include `year`) and one numeric variable for plotting purposes. The plot should create a boxplot of the numeric variable at the levels of the categorical variable. In the `aes()` include `weight = PWGTP` as this will treat each row appropriately (since some rows should be more prevalent than others!).

- Note: I have `get(cat_var)` and `get(num_var)` within the function as I passed the column names as strings. You may find this useful!

Investigation

Your web page should have a narrative going through your process of creating the above functions. Then you should test them. Add a section where you investigate something interesting from the data using your API function and plotting/summarizing functions. Be sure to use your plotting function and summary function in multiple ways (i.e. with different inputs) so we can see that they work!

Submission

For the submission link, you should give the link to your nicely rendered webpage!

Rubric for Grading (total = 100 points)

Item	Points	Notes
Data Processing Functions	35	Worth either 0, 2.5, 5, ... 35
Generic Functions	35	Worth either 0, 2.5, 5, ... 35
Narrative/Introduction/End Example/Good Coding/etc.	30	Worth either 0, 2.5, 5, ... 30

Notes on grading:

- For each item in the rubric, your grade will be lowered one level for each each error (syntax, logical, or other) in the code and for each required item that is missing or lacking a description.
- You should use Good Programming Practices when coding (see wolffware). If you do not follow GPP you can lose up to 50 points on the project.
- If the webpage doesn't work correctly you can lose up to 30 points.